

OmniAI Group of ZJU ACES Lab



Embodied-Navigator: Point, Think, Memorize, and Align for Efficient Navigation

Hongyan Feng^{*1}, Sunlai Chen^{*1}, Xuanyu Liu^{*1}, Miao Pan¹, Yangfan Xie¹, Yuxiang Cui², Zhongxiang Zhou², Rong Xiong², Wenqi Zhang¹, Jianwei Yin¹, Yueting Zhuang¹, Xuhong Zhang^{†1}

¹School of Software Technolog,Zhejiang University

²Zhejiang Humanoid Robot Innovation Center Co., Ltd.

^{*}Equal contribution, [†]Corresponding author

Although Large Vision-Language Models (VLMs) have significantly advanced embodied navigation, their direct deployment remains challenging, as existing methods often force VLMs into unnatural action spaces that misalign with their 2D pre-training priors, compounded by rigid reasoning schedules and inefficient memory management. To overcome these limitations, we propose **TAMP-Nav**, a unified framework for efficient embodied navigation. First, we introduce a **Pixel-to-3D Action Formulation** (*Point*) that reformulates navigation into 2D visual prompting. Specifically, the VLM merely selects 2D pixels, which are then projected into 3D coordinates for a low-level SLAM controller. This design naturally aligns embodied execution with the VLM’s inherent 2D visual capabilities. Second, we propose an integrated **Selective Reasoning and Anchor-Trajectory Memory** mechanism (*Think* and *Memorize*), which dynamically triggers Chain-of-Thought and retains high-fidelity memory only at critical nodes, compressing redundant trajectories into lightweight **Space-Time Indicators**, thereby preserving critical historical information and enhancing spatio-temporal perception. Finally, we design an efficient **Two-Level Alignment Paradigm** (*Align*) via Group Relative Policy Optimization (GRPO). By superimposing global outcome rewards with fine-grained process rewards, this dense supervision tightly aligns the agent’s cognitive planning with physical environmental feedback, endowing the model with adaptive reasoning capabilities. Experiments demonstrate that TAMP-Nav achieves state-of-the-art performance (e.g., 66.2% SR on R2R-CE) with high runtime and sample efficiency (requiring only 90k training trajectories).

Correspondence: zhangxuhong@zju.edu.cn

Date: August 2026

1 Introduction

Embodied Navigation requires agents to interpret natural language instructions and navigate through complex environments (Zhang et al. (2024c); Krantz et al. (2020); Ku et al. (2020)). While Large Vision-Language Models (VLMs) have vastly improved multi-modal understanding, directly deploying them in real-world Embodied Navigation exposes three critical technical bottlenecks:

The Geometric Gap in Action Formulation. Pioneering VLN methods have made great strides by mapping observations directly to low-level atomic actions (e.g., “turn left 30°”) (Zhang et al. (2024b,a); Cheng et al. (2024); Wei et al. (2025b)) or regressing 3D spatial coordinates (Wei et al. (2025a); Zhang et al. (2025b)). However, VLMs are predominantly pre-trained on 2D image-text pairs. Forcing them to implicitly learn complex 3D geometric transformations leads to spatial hallucinations and low sample efficiency (Zhang et al. (2025a); Stogiannidis et al. (2025)).

The Dilemma Between Navigation Performance and Inference Efficiency. Integrating Chain-of-Thought (CoT) (Wei et al. (2022)) has enhanced the high-level planning capabilities of navigation models. However, existing models typically trigger reasoning at **every step or predefined intervals** (Lin et al. (2025); Song et al. (2023); Liu et al. (2026)). While dense reasoning improves decision-making quality, it also introduces severe inference latency. Ultimately, such rigid cognitive designs fail to strike an optimal balance between runtime efficiency and navigation performance.

The Loss of Key Information and Spatio-Temporal Perception in Long-Horizon Memory. Existing navigation memory mechanisms struggle to manage long-horizon histories effectively. Retaining full historical visual features inevitably leads to attention dilution and memory overflow; conversely, indiscriminately discarding past observations (Zhang et al. (2024b), Wei et al. (2025a)) or forgetting curves (Zhang et al. (2025b)) may result in the loss of critical information. More importantly, because existing architectures often lack explicit spatio-temporal coordinates to organize this historical information, agents struggle to maintain a clear comprehension of their past trajectories.

To break these bottlenecks, we propose **TAMP-Nav** (Think, Align, Memorize, and Point).

First, to bridge the geometric gap, we introduce a **Pixel-to-3D Action Space** (*Point*). To bypass complex 3D reasoning and directly leverage 2D image-text priors, the agent acts as a “pointer,” directly outputting 2D pixel coordinates on the egocentric visual observations, which are then projected into 3D and executed by a local SLAM controller (Yang et al. (2022); Xu et al. (2022)).

Second, for efficient reasoning and memorization, we propose an integrated **Selective Reasoning and Anchor-Trajectory Memory** (*Think and Memorize*). TAMP-Nav discards rigid step-by-step CoT. Instead, the model dynamically triggers deep CoT only at critical topological nodes, which are subsequently stored as high-fidelity **Explicit Anchors**. Conversely, routine path intervals are bypassed without reasoning and compressed into lightweight **Space-Time Indicators**, strictly controlling the memory budget while preserving key information and spatio-temporal perception.

Crucially, to more efficiently enhance the model’s navigation and adaptive reasoning capabilities, we design a **Two-Level GRPO** framework (*Align*) featuring an innovative multi-branch rollout mechanism. Moving beyond strict expert-forcing, our method performs autonomous group rollouts: the agent samples a diverse set of candidate actions at each decision node and unrolls them into multiple complete, distinct trajectories. Across these diverse rollouts, we seamlessly superimpose **global outcome rewards** (task success, trajectory efficiency via SPL, and reasoning density constraints) with explicit, **fine-grained process rewards** (target approach metrics, collision avoidance, stop action correctness, reasoning value assessment, and format adherence). By evaluating group-relative advantages at both the trajectory and step levels, this dual-reward architecture provides dense supervision that tightly aligns the agent’s internal cognitive planning with physical environmental feedback. Through this alignment process, the agent autonomously acquires the meta-cognitive ability to trigger CoT reasoning in a state-dependent manner.

Our main contributions are summarized as follows:

(1) Vision-Centric Action Formulation(Point): We introduce a Pixel-to-3D action space that decouples high-level visual prompting from low-level execution. This effectively bridges the geometric gap, enabling the agent to leverage 2D VLM pre-training priors for 3D embodied navigation.

(2) Efficient Cognitive Architecture (Think and Memorize): We propose an integrated mechanism that couples dynamic selective reasoning with Anchor-Trajectory Memory. This resolves the trade-off between inference efficiency and navigation performance while enabling efficient history storage through explicit spatio-temporal encoding and key-node memory.

(3) Efficient Two-Level Alignment Paradigm (Align): We design a two-level GRPO framework that superimposes global outcome rewards with fine-grained process rewards. This dense supervision tightly aligns the agent’s cognitive planning with physical environmental feedback, endowing the model with adaptive reasoning capabilities and achieving state-of-the-art performance (e.g., 66.2% SR on R2R-CE) with high sample efficiency (requiring only 90k training trajectories).

2 MultiNav-CoT Dataset

To endow TAMP-Nav with high-level logical reasoning and spatial understanding, we construct **MultiNav-CoT**, a curated dataset containing 90k trajectories. Derived from VLN-CE (Krantz et al. (2020)), these samples are filtered to exclude rendering artifacts (e.g., black images) and Gemini-assessed low-quality instructions. MultiNav-CoT features **selective Chain-of-Thought** annotations to supervise sparse reasoning and solve the RL cold-start problem. To generate this high-value reasoning, we developed a CoT engine comprising **Key Node Mining** and **Structured Generation**.

(1) Spatiotemporal Key Node Mining To determine exactly *when* the agent should trigger deep reasoning, we propose a three-step Spatiotemporal Key Node Mining strategy (detailed pseudocode is provided in Appendix C). First, we compute an importance score

$$S(t) = S_{sem}(t) + S_{vis}(t) \quad (1)$$

For each frame, S_{sem} captures the semantic relevance between the current visual observation and the language instruction via CLIP similarity (Radford et al. (2021)), while S_{vis} measures scene transitions based on visual feature differences (Dosovitskiy et al. (2020); detailed formulations are provided in Appendix C). Second, we apply a **distance-based greedy filtering** to the score-sorted frames, iteratively selecting high-scoring nodes while discarding candidates within a minimum spatial distance D_{min} to ensure reasoning sparsity. Third, to maintain global topological connectivity, we perform temporal padding by inserting the highest-scoring intermediate frame whenever the spatial gap between consecutive nodes exceeds D_{max} . Ultimately, this dynamic selection yields a sparse set of key nodes comprising roughly 30% of the trajectory, serving as anchors for CoT generation.

(2) Structured CoT Generation: We adopt a multi-stage prompt engineering pipeline based on **Gemini 2.5 flash** (Comanici et al. (2025)) to construct the reasoning annotations. Rather than prompting the model to generate the entire reasoning chain in a single pass, we decompose the generation into three independent sub-tasks: **Task Phase Localization**, **Current Observation Analysis**, and **Future Action Reasoning**. Generating these components separately reduces the cognitive load of a single LLM query, minimizing hallucinations and ensuring high stability and quality of the CoT. Subsequently, a final fusion step synthesizes these outputs into a unified, coherent narrative. This multi-stage approach establishes a strict logical flow that tightly couples visual perception with long-horizon planning. Detailed prompts are provided in Appendix G.

3 Method

To seamlessly bridge high-level reasoning with low-level execution while achieving high sample and inference efficiency, we propose **TAMP-Nav**, as illustrated in Figure 1. Our framework operates through a continuous interaction loop. At each navigation step, the agent first integrates current visual observations with an **Anchor-Trajectory Memory** to maintain long-horizon spatial awareness. Then, it autonomously decides whether to trigger **Selective Reasoning** to analyze complex scenarios. Following this, the model employs a **Pixel-to-3D Action Formulation** to directly predict a pixel waypoint, which is then converted into 3D spatial coordinates and passed to the low-level planner for execution. The entire decision-making process is optimized via **Two-Level GRPO**, aligning the agent to maximize navigation success while minimizing computational overhead.

3.1 Pixel-to-3D Action Formulation

TAMP-Nav acts as a visual pointer. At step t , the agent receives four egocentric observations $V_t = \{v_{t,1}, v_{t,2}, v_{t,3}, v_{t,4}\}$ to cover a 360° field of view. The VLM first selects the optimal view $v_{t,i}$ and subsequently outputs a 2D pixel $a_t = (u, v)$ pointing to the target waypoint. This pixel is projected into a local 3D point P_t :

$$P_t = D_{t,i}(u, v) \cdot K^{-1}[u, v, 1]^T \quad (2)$$

where $D_{t,i}$ denotes the depth map and K represents the camera intrinsic matrix. This 3D coordinate is then transformed into the world coordinate system and dispatched to a low-level SLAM controller, which

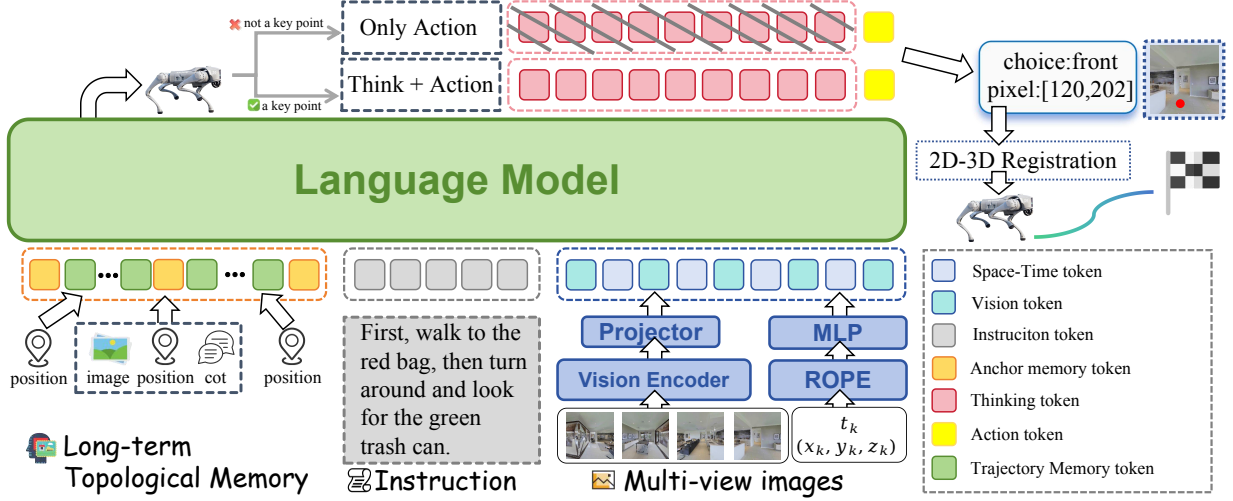


Figure 1 The architecture of TAMP-Nav. Given visual-textual inputs, the agent compresses long-horizon history into an Anchor-Trajectory Memory, autonomously triggering reasoning at critical nodes. As a visual pointer, it selects the optimal multi-camera view and predicts a 2D pixel, which is projected into 3D for SLAM execution. Finally, the policy is optimized via Two-Level GRPO.

handles local movement to reach P_t . This formulation relieves the VLM from learning complex geometric transformations, allowing it to focus entirely on visual-semantic grounding.

In simulation environments, the ground-truth depth map $D_{t,i}$ is directly accessible. For real-world deployment, we directly obtain the depth map using a depth camera. We evaluate robustness to depth inaccuracies in Section 4.4. Because navigation operates as a closed-loop interactive process, subsequent observations can correct some geometric deviations caused by single-step depth noise.

3.2 Anchor-Trajectory Memory

Traditional VLN memory mechanisms often face a dilemma between “full storage leading to context overflow” and “sparse sampling leading to information loss.” We argue that while non-critical nodes are redundant in high-level semantics, they carry indispensable spatiotemporal transition information. Based on this, the memory is constructed via two components: **Explicit Anchors** (i.e., the specific decision nodes where CoT reasoning is triggered) and **Space-Time Indicators**

Designed to inject precise geometric positions (x, y, yaw) and temporal (t) causality into the context stream, the STI utilizes a feature fusion approach. Specifically, we separately encode spatial coordinates, temporal indices, and angular information using 2D and 1D Rotary Position Embeddings (RoPE) (Su et al. (2024)). These embeddings are then concatenated and projected through a Multi-Layer Perceptron (MLP) to form a unified STI token:

$$E_{STI}(t, x, y, yaw) = \text{MLP}\left(\left[\text{RoPE}_{2D}(x, y); \text{RoPE}_{1D}(t); \text{RoPE}_{2D}(\sin(yaw), \cos(yaw))\right]\right) \quad (3)$$

where $[\cdot]$ denotes the concatenation operation. Notably, instead of directly encoding the scalar yaw , we map it to its continuous trigonometric representation $(\sin(yaw), \cos(yaw))$ to prevent the discontinuity at the $0^\circ/360^\circ$ boundary, ensuring smooth orientation encoding. The MLP projects these concatenated high-frequency trigonometric features into the model’s native hidden dimension. The resulting E_{STI} is then added into the input sequence as a single token. This explicitly embeds the absolute spatio-temporal state into a fixed-length space, providing the VLM with dense geometric and temporal awareness for navigation (detailed analysis in E).

Based on this geometric foundation, the memory is constructed via two heterogeneous components: **Explicit Anchors** and **Space-Time Indicators**. When the selective reasoning mechanism triggers a Chain-of-Thought

(CoT) at a critical topological node t_k , the state is stored as a triplet:

$$A_k = \langle M_{STI}^{(k)}, M_{vis}^{(k)}, M_{state}^{(k)} \rangle \quad (4)$$

Here, $M_{STI}^{(k)} = E_{STI}(t_k, x_k, y_k, yaw_k)$ provides the explicit spatiotemporal coordinate; $M_{vis}^{(k)} = v_{t_k}$ retains high-fidelity raw visual features to support pixel-level loop closure detection; and $M_{state}^{(k)}$ acts as a ‘‘Planning Beacon’’ by storing CoT to represent the task phase localization, serving as a semantic summary of navigation progress. For the redundant path interval $\mathcal{T}$ between two anchors, the model discards redundant visual features and retains only a pure spatiotemporal stream:

$$T_k = [E_{STI}(t, x_t, y_t, yaw_t) \mid t \in \mathcal{T}] \quad (5)$$

This mechanism allows the memory to maintain full-resolution movement trajectories in non-critical intervals, effectively preventing topological information loss while concentrating expensive visual perception overhead on critical anchors.

At each decision step, the dynamic input context C_t concatenates a Working Memory (comprising two uniformly sampled frames between the current step and the most recent reasoning anchor) with the Long-term Topological Memory (the reconstructed sequence $M_{long} = \{A_1, T_1, \dots, A_m, T_m\}$).

3.3 Two-Level GRPO Paradigm

To ensure the model possesses robust priors before the RL phase, we first perform a brief Supervised Fine-Tuning (SFT) (Ouyang et al. (2022)) cold start on the MultiNav-CoT dataset, using Qwen2.5-VL-7B (Bai et al. (2025)) as base model. After this cold start, we employ **Group Relative Policy Optimization (GRPO)** (Shao et al. (2024)) for further fine-tuning. We designed a **Dual Reward System**, combined with group-based relative optimization, to simultaneously enhance fine-grained operational skills and long-range planning capabilities.

3.3.1 Local Step Rewards

Local step rewards $R_{local}^{(t)}$ guide the agent to make logical, safe, and timely reasoning decisions at each step. We define five explicit components: **Target Approach Reward** ($r_{app}^{(t)}$) encourages continuous movement toward the target; **Collision Avoidance Reward** ($r_{coll}^{(t)}$) ensures physical traversability; **Stop Action Reward** ($r_{stop}^{(t)}$) provides supervision for termination; **Reasoning Value Reward** ($r_{rea}^{(t)}$) evaluates the actual utility of reasoning by correlating CoT indicators with approach scores; and **Format Adherence Reward** ($r_{fmt}^{(t)}$) ensures valid spatial constraints and JSON formatting. The Total Local Reward is defined as (advantage calculations and hyperparameter are in Appendix A):

$$R_{local}^{(t)} = \lambda_1 r_{app}^{(t)} + \lambda_2 r_{coll}^{(t)} + \lambda_3 r_{stop}^{(t)} + \lambda_4 r_{rea}^{(t)} + \lambda_5 r_{fmt}^{(t)} \quad (6)$$

3.3.2 Global Trajectory Rewards

The global reward evaluates the holistic quality of the completed trajectory, aggregating three metrics: **Task Success Reward** (r_{suc}) is the primary driver relaxing strict success to oracle boundaries; **Trajectory Efficiency Reward** (r_{spl}) optimizes navigation efficiency via SPL (Anderson et al. (2018)); and **Reasoning Density Reward** (r_{den}) provides a non-negative efficiency bonus that discourages excessive CoT generation. The Total Global Reward is defined as (advantage calculations and hyperparameter are in Appendix A):

$$R_{global} = \omega_1 r_{suc} + \omega_2 r_{spl} + \omega_3 r_{den} \quad (7)$$

This reward design encourages the agent to minimize cognitive overhead while ensuring task success, naturally guiding it to identify critical decision nodes and eliminate redundant reasoning.

3.3.3 Two-Level Policy Optimization

Unlike previous methods that rely on step-by-step expert trajectories, our GRPO paradigm (Figure 2) encourages autonomous exploration by superimposing advantages from two hierarchical levels of rollouts. For a given instruction q , we generate a group of $G = 8$ complete navigation trajectories. To construct each

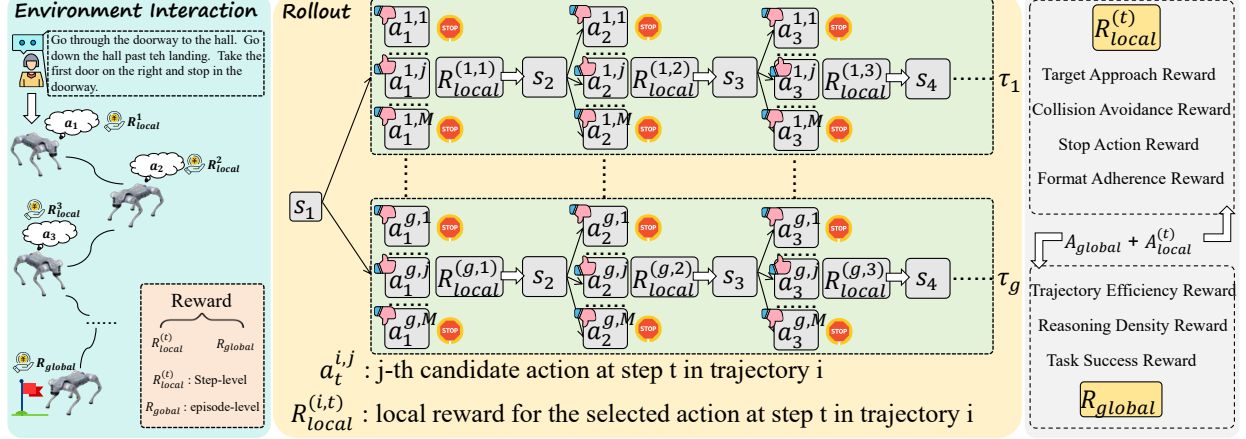


Figure 2 The Two-Level GRPO paradigm. TAMP-Nav superimposes trajectory-level rollouts (competing for global navigation success) with step-level candidate rollouts (exploring diverse 2D visual-spatial actions via VLM temperature sampling).

trajectory, at each decision step t , the policy generates a candidate set $\mathcal{A}_t = \{a_1, a_2, \dots, a_M\}$ containing $M = 4$ actions via temperature-based sampling based on the current context C_t . We then evaluate these candidates using the local step reward $R_{local}^{(t)}$.

To ensure a healthy variance of trajectory qualities for effective gradient updates, we introduce an annealed guided sampling strategy during early RL stages. Specifically, the probability of actually selecting candidate $a_i \in \mathcal{A}_t$ to execute and continue the trajectory is formulated as:

$$P_{select}(a_i) = \frac{\exp(\beta_k \cdot R_{local}^{(t)}(a_i))}{\sum_{j=1}^M \exp(\beta_k \cdot R_{local}^{(t)}(a_j))} \quad (8)$$

where β_k is a guidance coefficient at the global RL training iteration k , explicitly defined as an exponential decay schedule $\beta_k = \beta_0 \cdot \alpha^k$ with initial value $\beta_0 > 0$ and decay rate $\alpha \in (0, 1)$. During early stages ($\beta_k > 0$), this explicitly weights the probability of selecting a candidate by its local reward. As training progresses and $\beta_k \rightarrow 0$, $P_{select}(a_i)$ approaches a uniform distribution $1/M$. Once the G trajectories are generated, we compute A_{global} and A_{local} via separate standardizations. Specifically, A_{global} is obtained by Z-score standardizing the global trajectory rewards across the G rollouts. Concurrently, at each decision step, the local advantage $A_{local}^{(t)}$ for the actually executed action a_i is derived by Z-score standardizing its local reward $R_{local}^{(t)}(a_i)$ against the M candidates in $\mathcal{A}_t$. By independently normalizing both advantages to a standard normal distribution ($\mathcal{N}(0, 1)$), we align their statistical scales to facilitate a more balanced integration into the superimposed total advantage $A_S^{(t)}$:

$$A_S^{(t)} = A_{global} + A_{local}^{(t)} \quad (9)$$

The policy is updated via the standard GRPO loss formulas provided in Appendix A.

4 Experiments

4.1 Experimental Setup

Evaluation Strategy and Metrics. To rigorously assess TAMP-Nav, we conduct comprehensive experiments across three dimensions: (1) Standard benchmarking on VLN-CE datasets; (2) In-depth analytical experiments on emergent cognitive abilities and training efficiency; and (3) Ablation studies on key architectural components. We report performance using standard metrics: Success Rate (SR), Oracle Success Rate (OSR), Success weighted by Path Length (SPL), normalized Dynamic Time Warping (nDTW), and Navigation Error (NE).

Table 1 Comparison of different methods on R2R-CE and RxR-CE validation unseen splits.

Method	R2R-CE Val-Unseen				RxR-CE Val-Unseen			
	NE↓	OS↑	SR↑	SPL↑	NE↓	SR↑	SPL↑	nDTW↑
HPN+DN* Krantz et al. (2021)	6.31	40.0	36.0	34.0	-	-	-	-
CMA* Hong et al. (2022)	6.20	52.0	41.0	36.0	8.76	26.5	22.1	47.0
GridMM* Wang et al. (2023a)	5.11	61.0	49.0	41.0	-	-	-	-
ETPNav* An et al. (2024)	4.71	65.0	57.0	49.0	5.64	54.7	44.8	61.9
ScaleVLN* Wang et al. (2023b)	4.80	-	55.0	51.0	-	-	-	-
InstructNav Long et al. (2024)	6.89	-	31.0	24.0	-	-	-	-
R2R-CMTP Chen et al. (2021)	7.90	38.0	26.4	22.7	-	-	-	-
LAW Raychaudhuri et al. (2021)	6.83	44.0	35.0	31.0	10.90	8.0	8.0	38.0
CM2 Georgakis et al. (2022)	7.02	41.5	34.3	27.6	-	-	-	-
WS-MGMap Chen et al. (2022)	6.28	47.6	38.9	34.3	-	-	-	-
ETPNav + FF Wang et al. (2024)	5.95	55.8	44.9	30.4	8.79	25.5	18.1	-
Seq2Seq Krantz et al. (2020)	7.77	37.0	25.0	22.0	12.10	13.9	11.9	30.8
CMA Krantz et al. (2020)	7.37	40.0	32.0	30.0	-	-	-	-
VLN-R1 Qi et al. (2025)	7.0	41.2	30.2	21.8	9.1	22.7	17.6	-
NaVid Zhang et al. (2024b)	5.47	49.1	37.4	35.9	-	-	-	-
MapNav Zhang et al. (2025c)	4.93	53.0	39.7	37.2	-	-	-	-
NaVILA Cheng et al. (2024)	5.22	62.5	54.0	49.0	6.77	49.3	44.0	58.8
UniNaVid Zhang et al. (2024a)	5.58	53.3	47.0	42.7	6.24	48.7	40.9	-
StreamVLN Wei et al. (2025b)	4.98	64.2	56.9	51.9	6.22	52.9	46.0	61.9
NavFoM Zhang et al. (2025b)	4.61	72.1	61.7	55.3	4.74	64.4	56.2	65.8
DualVLN Wei et al. (2025a)	4.05	70.7	64.3	58.5	4.58	61.4	51.8	70.0
TAMP-Nav(only SFT)	4.88	62.0	55.7	50.3	6.10	52.4	46.2	62.1
TAMP-Nav	3.85	74.5	66.2	58.8	4.32	65.7	56.9	72.4

Implementation Details. Both simulation and real-world experiments utilize a context window of 4,096 tokens. Training, evaluation, and real-world deployment were all conducted on NVIDIA A800 GPUs. Detailed training configurations and hyperparameters are provided in Appendix H.

4.2 Main Results

Performance on VLN-CE Benchmarks. As shown in Table 1, TAMP-Nav achieves State-of-the-Art (SOTA) performance across all metrics on the R2R-CE and RxR-CE validation unseen splits (66.2% on R2R-CE Val-Unseen and 65.7% on RxR-CE Val-Unseen). A representative qualitative example of the agent’s execution in the simulation environment is provided in Appendix F.

Beyond navigation accuracy, TAMP-Nav demonstrates high data and computational efficiency. It achieves competitive results using only 90k training trajectories (700k interactions), significantly fewer than DualVLN (763k trajectories) and NavFoM (3.37M interactions). We attribute this sample efficiency to our dense-reward RL optimization, which improves the success rate on R2R-CE from 55.7% (SFT-only) to 66.2%, proving more effective than pure supervised learning in internalizing complex logic from limited data. Computationally, the Pixel-to-3D action paradigm reduces average interaction steps to 9 per trajectory, whereas open-source state-of-the-art models such as DualVLN and StreamVLN require approximately 30 steps. Benefiting from this compacted interaction horizon, sparse reasoning strategy, and the lightweight memory mechanism, TAMP-Nav achieves highly efficient inference: it averages 16.58s per task on a single A800 GPU, significantly faster than StreamVLN (37.47s) and DualVLN (41.46s).

Performance on Long-Horizon Tasks. To further investigate TAMP-Nav’s performance stability over extended distances, we evaluate TAMP-Nav on a subset of long-horizon trajectories from the R2R-CE and RxR-CE

val-unseen splits. This subset comprises 5927 trajectories where the expert path length exceeds 12.5 meters (equivalent to over 50 atomic forward actions). As shown in Table 4, TAMP-Nav (AT-Mem) significantly outperforms strong baselines, achieving a Success Rate (SR) of 49.8%, which is a substantial improvement over StreamVLN (30.9%) and DualVLN (41.9%). Figure 4 further breaks down performance across varying trajectory lengths.

4.3 Emergent Reasoning-on-Demand

During the inference phase, TAMP-Nav autonomously determines *when* to trigger selective reasoning. To verify this capability, we analyze the spatial distribution of CoT triggers alongside comparative intervention experiments across different trigger strategies.

As illustrated in Figure 3, we recorded the locations where the model generated `<think>` tokens and applied Gaussian Kernel Density Estimation (KDE) to visualize their spatial distribution. The heatmaps indicate that the RL-trained TAMP-Nav concentrates reasoning at critical topological nodes, such as crossroads, doorways, and near target objects. Quantitatively, the proportion of reasoning steps allocated to straight corridors decreases from 38% in the SFT model to 11% after RL alignment. This indicates that the agent learns to suppress redundant CoT in trivial intervals, prioritizing computation for complex decision junctures.

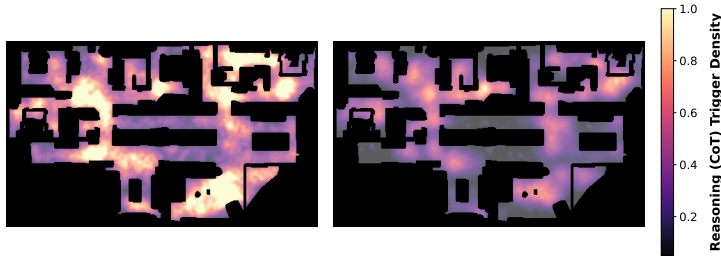


Figure 3 Spatial Heatmap of Reasoning (CoT) Triggers. Comparison between the SFT model (left) and the RL-aligned TAMP-Nav (right).

Table 2 Intervention Analysis. Comparing navigation success (SR) and reasoning density (CoT Ratio) on R2R-CE Val-Unseen.

Strategy	CoT Ratio	SR
Dense CoT	100%	66.8%
Fixed (1/3)	36.2%	60.1%
Auto (Ours)	26.3%	66.2%

Table 2 evaluates this emergent timing. Remarkably, TAMP-Nav nearly matches the *Dense CoT* upper bound (66.8% vs. 66.2% SR) with only a 26.3% CoT ratio, significantly outperforming the *Fixed-Interval* baseline. Supported by Figure 3, these results confirm our RL formulation effectively incentivizes the agent to bypass redundant thinking, triggering reasoning at optimal moments.

4.4 Robustness to Depth Uncertainty

To assess robustness against sensor inaccuracies, we inject multiplicative Gaussian noise into the depth of the selected pixel:

$$\tilde{d}_t = D_t(u, v) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(1, \sigma^2) \quad (10)$$

where σ is the relative noise intensity. To mimic low-level collision avoidance, any perturbed waypoint in non-navigable regions is snapped to the nearest valid point.

Under the tested multiplicative-noise protocol, SR decreases by 2.8 points at $\sigma = 0.2$ (16% relative error), indicating robustness to moderate waypoint-depth perturbations.

4.5 Ablation Studies

4.5.1 Controlled Component Analysis.

We conduct block-wise ablations on the R2R-CE and RxR-CE val-unseen splits. All variants are initialized from Qwen2.5-VL-7B and use the same sensing inputs, data split, fixed non-learned SLAM controller, and evaluation protocol. Each block varies one component while holding the remaining configuration fixed; comparisons are therefore made within each block.

Table 3 Success Rate (SR) under varying multiplicative depth noise levels (σ).

Noise σ	0	0.05	0.1	0.2
SR (%)	66.2	65.9	65.3	63.4
Δ SR	-	-0.3	-0.9	-2.8

Table 4 Performance on long-horizon tasks and memory ablation study.

Model	SR ($\uparrow$)
StreamVLN	30.9
DualVLN	41.9
TAMP-Nav(US)	40.5
TAMP-Nav(Full)	42.4
TAMP-Nav(w/o STI)	45.6
TAMP-Nav(AT-Mem)	49.8

Table 5 Controlled component-wise ablations under matched budgets. R2R-CE reports NE/OS/SR/SPL; RxR-CE reports NE/SR/SPL/nDTW.

Block	Variant	R2R-CE: NE / OS / SR / SPL	RxR-CE: NE / SR / SPL / nDTW
Action (SFT only)	NavFoM-style metric waypoint	7.18 / 42.0 / 30.9 / 24.7	8.47 / 27.2 / 20.6 / 43.1
	Pixel-to-3D	4.88 / 62.0 / 55.7 / 50.3	6.10 / 52.4 / 46.2 / 62.1
GRPO	Global trajectory advantage only	4.52 / 67.0 / 59.3 / 52.1	5.30 / 58.4 / 49.7 / 65.5
	Two-Level without annealed guidance	4.12 / 71.5 / 63.4 / 56.0	4.72 / 62.6 / 53.8 / 69.4
	Full Two-Level GRPO	3.85 / 74.5 / 66.2 / 58.8	4.32 / 65.7 / 56.9 / 72.4
Memory	Uniform sampling	4.43 / 68.2 / 60.8 / 53.6	5.03 / 59.4 / 50.8 / 66.8
	Full history	4.29 / 69.4 / 61.9 / 54.7	4.87 / 61.1 / 51.8 / 68.0
	AT-Mem without STI	4.08 / 71.4 / 63.6 / 56.1	4.59 / 62.4 / 52.7 / 69.8
	Full AT-Mem	3.85 / 74.5 / 66.2 / 58.8	4.32 / 65.7 / 56.9 / 72.4
Reasoning	Dense CoT	3.79 / 75.1 / 66.8 / 59.3	4.25 / 66.1 / 57.4 / 72.8
	Fixed (1/3)	4.46 / 67.6 / 60.1 / 52.8	5.10 / 58.9 / 50.0 / 65.9
	Auto	3.85 / 74.5 / 66.2 / 58.8	4.32 / 65.7 / 56.9 / 72.4

The SFT-only action block uses 90k trajectories, approximately 700k interactions, and one training epoch without GRPO. Replacing Pixel-to-3D with a NavFoM-style metric-waypoint output while keeping the remaining architecture fixed reduces SR by 24.8/25.2 points on R2R-CE/RxR-CE. In the GRPO block, local advantages improve SR by 4.1/4.2 points over trajectory-level advantage alone, and annealed guidance contributes a further 2.8/3.1 points. Full AT-Mem improves over full-history retention by 4.3/4.6 points, while removing STI decreases SR by 2.6/3.3 points. Auto reduces R2R-CE CoT calls by 73.7% relative to Dense CoT with a 0.6-point SR decrease; the corresponding SR decrease on RxR-CE is 0.4 points.

4.5.2 Influence of CoT Supervision.

We evaluate the effect of the CoT supervision source under a matched SFT budget. All settings share the same action labels, training trajectories, navigation policy architecture, and one-epoch schedule; only the reasoning annotations differ.

Table 6 Effect of the CoT supervision source under a matched SFT budget.

SFT supervision	R2R-CE: NE / OS / SR / SPL	RxR-CE: NE / SR / SPL / nDTW
Action only (no CoT)	5.31 / 57.0 / 50.6 / 45.0	6.71 / 46.1 / 39.8 / 56.2
Qwen2.5-VL-7B CoT	5.46 / 55.7 / 49.4 / 43.1	6.83 / 44.8 / 38.2 / 54.9
GLM-4.5V-108B CoT	4.99 / 60.8 / 54.4 / 48.9	6.24 / 51.2 / 44.9 / 60.8
Gemini 2.5 Flash CoT	4.88 / 62.0 / 55.7 / 50.3	6.10 / 52.4 / 46.2 / 62.1

Qwen2.5-VL-7B supervision performs below the Action-only setting on both benchmarks. Inspection of its annotations reveals frequent spatial contradictions, incorrect task-phase identification, and inconsistencies between the rationale and waypoint. GLM-4.5V-108B substantially improves over the smaller teacher and remains within 1.3/1.2 SR points of Gemini 2.5 Flash on R2R-CE/RxR-CE. We further conduct a human quality assessment in which two annotators independently evaluate 200 randomly sampled CoTs from each of

the two stronger teachers for instruction phase, visual-spatial consistency, rationale-waypoint consistency, and completeness. Gemini and GLM achieve pass rates of 94.0% and 91.5%, respectively. These results show that downstream navigation performance is sensitive to the spatial consistency of CoT supervision, while the gap between the strong open-weight and proprietary teachers is modest.

4.5.3 Effectiveness of Memory Mechanisms.

As shown in Table 4, we compare our Anchor-Trajectory Memory with traditional history management strategies and ablate its core component. **TAMP-Nav(US)** and **TAMP-Nav(Full)** represent conventional memory designs relying solely on raw visual features via uniform sampling or full history retention. AT-Mem effectively overcomes the critical information loss inherent in Uniform Sampling and the severe attention dilution suffered by Full History. Furthermore, removing **Space-Time Indicators (w/o STI)** while preserving other AT-Mem components drops performance from 49.8% to 45.6% SR. This demonstrates that while visual anchors provide semantic grounding, lightweight STI tokens are essential for maintaining long-horizon spatial memory.

To examine how these memory mechanisms behave as trajectories grow longer, Figure 4 groups the combined R2R-CE and RxR-CE long-horizon subset by expert trajectory length. Each atomic forward action corresponds to a 0.25-meter movement; bins span from 50 to more than 210 actions in intervals of 10. The bars show the number of trajectories in each bin, while the curves report the SR of each method.

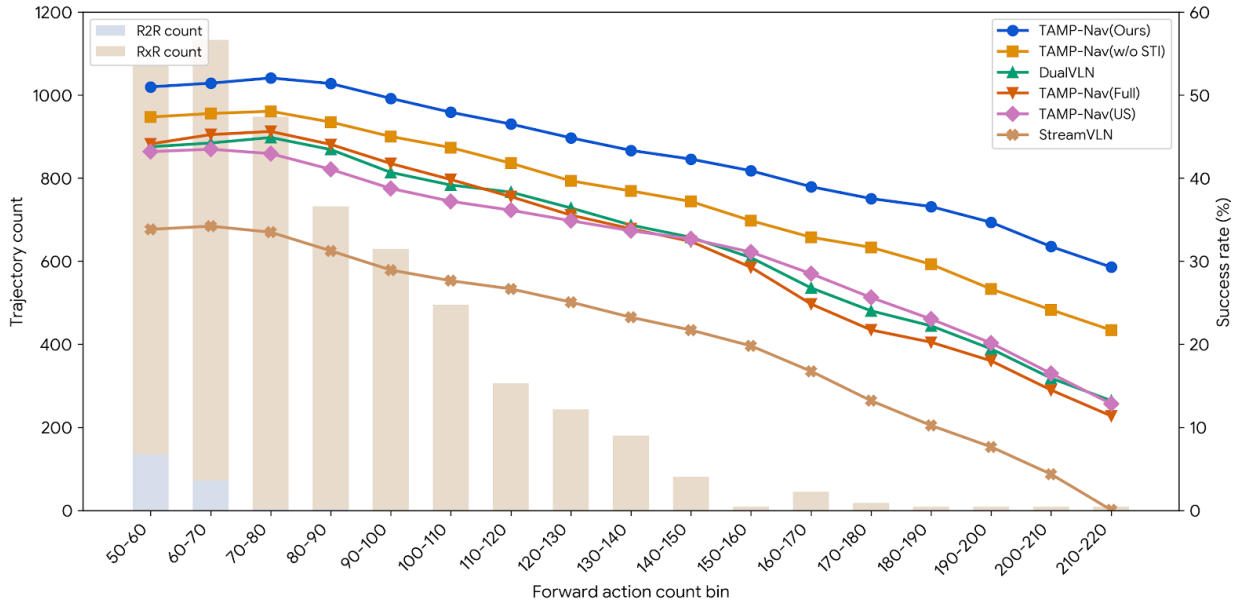


Figure 4 Performance distribution on long-horizon navigation tasks. The bars show the trajectory count in each length bin, and the curves show the Success Rate (SR) of the evaluated methods as trajectory length increases.

Three trends emerge. First, TAMP-Nav maintains a performance margin over StreamVLN and DualVLN across all length bins; for example, in the 150–160-action bin, TAMP-Nav achieves approximately 40% SR, whereas StreamVLN drops to approximately 20%. Second, under the same fixed token budget, Uniform Sampling spans the trajectory using a budget-dependent sampling interval, whereas Full History retains the most recent frames after the budget is exceeded. Uniform Sampling increasingly loses critical observations as paths lengthen, while Full History suffers from attention dilution. In contrast, AT-Mem preserves critical visual anchors and compresses intermediate motion into lightweight trajectory tokens. Third, the gap between full TAMP-Nav and **TAMP-Nav (w/o STI)** widens on longer trajectories (e.g., beyond 120 actions), indicating that STI tokens help preserve geometric connectivity over extended execution.

4.5.4 Effectiveness of the Two-Level GRPO Framework

To evaluate our RL design, we analyze the learning dynamics of TAMP-Nav under different GRPO configura-

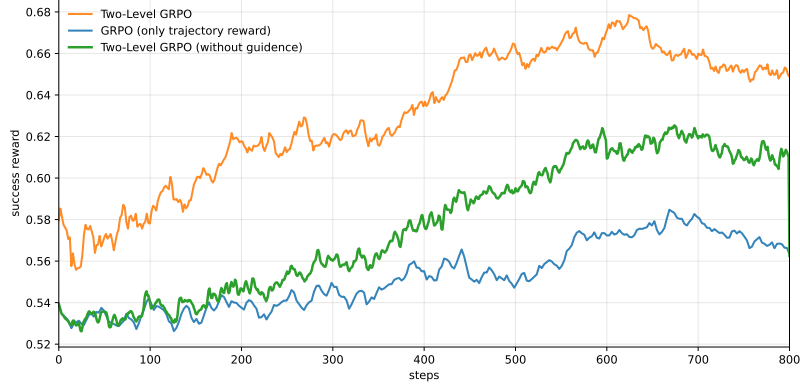


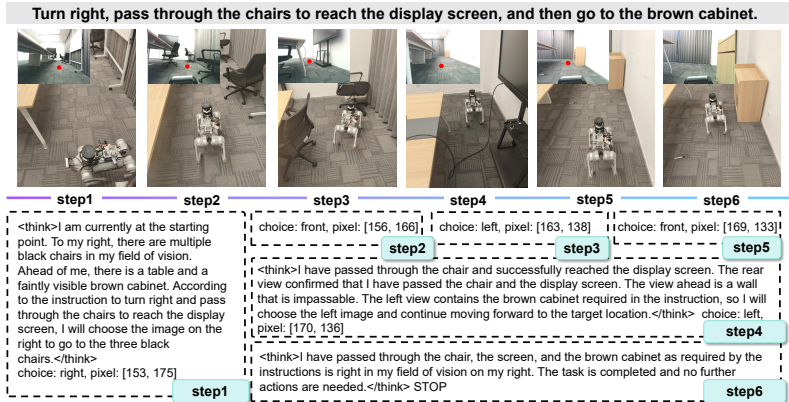
Figure 5 Learning curves of success rewards under different GRPO configurations. The plot illustrates the training progress over 800 steps for three variants: the full Two-Level GRPO, the version without annealed guided sampling (“without guidance”), and the standard GRPO using only trajectory-level rewards (“only trajectory reward”).

tions. Figure 5 compares the success reward across 800 training steps for standard GRPO (trajectory rewards only), Two-Level GRPO without guidance, and the full framework.

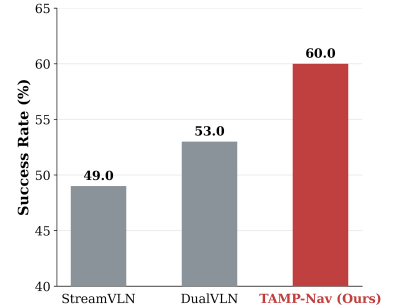
Overcoming Sparse Feedback. The baseline model trained solely with trajectory-level rewards exhibits training instability. In long-horizon navigation, the delay of terminal rewards makes it difficult for the model to associate final outcomes with specific intermediate reasoning or spatial decisions. This results in slower learning and convergence at a 0.59 reward. By superimposing local step advantages, our Two-Level GRPO provides immediate signals that stabilize intermediate state optimization, lifting the reward to 0.64 even without guided sampling.

Impact of Guided Sampling. The full Two-Level GRPO, which incorporates annealed guided sampling, achieves faster convergence and a higher final performance (0.68). This indicates that in early RL stages, purely random temperature sampling can produce rollouts with limited informative value. By weighting candidate selection based on local step rewards, guided sampling increases the density of high-quality trajectories in early samples, thereby accelerating policy updates and improving exploration efficiency.

4.6 Real-World Deployment



(a) Visualization of real-world experiments



(b) Success rate of real-world experiments

Figure 6 Real-world deployment results. (a) Visualization of the execution trajectory. (b) Success rate comparison.

Evaluation protocol. The evaluation covers five task categories: meeting room, laboratory, hall, cross-area, and outdoor navigation. Each category contains four fixed natural-language instructions, and each instruction

is repeated five times. Each method is therefore evaluated in 100 valid trials over the same 20 instructions, start positions, and goal positions, including 80 indoor and 20 outdoor trials. The complete evaluation contains 300 valid trials across the three methods. The surveyed feasible reference routes range from 8.88 to 44.74 meters, with a mean length of 24.55 meters.

Outcome and trial-validity criteria. Each high-level model request is limited to 5 seconds, and the low-level system is allowed 30 seconds to execute each high-level action. A trial is considered successful only if the robot autonomously stops within 1.5 meters of the surveyed target without human navigation or control intervention. Every trial that enters the navigation process normally is included in the evaluation. A trial is counted as a failure if it does not meet the success criterion, exceeds either time limit, or requires the safety operator to activate the hardware emergency stop or assume control because of an imminent collision, fall, or other safety risk. A trial is declared invalid and repeated under the same setting only when the execution logs confirm a robot mechanical fault or experimental equipment fault unrelated to the navigation execution chain. Any problem arising after a normal start from visual perception, model decision-making, localization, planning, control, or stopping is retained as a failure and is neither excluded nor retried.

Comparison scope. All methods share the robot platform, tasks, start and goal positions, time limits, and success criterion, while retaining their recommended sensing and low-level control mechanisms. The physical results therefore compare the complete deployed systems under their respective configurations.

As shown in Figure 6b, without any real-robot fine-tuning, TAMP-Nav achieved a 60.0% Success Rate, outperforming StreamVLN (49.0%) and DualVLN (53.0%). Furthermore, Figure 6a illustrates a representative execution where the agent effectively follows instructions and exhibits reasoning-on-demand behavior. Additional trajectory examples are provided in Appendix D.

5 Limitations

TAMP-Nav depends on depth information for Pixel-to-3D projection, STI encoding, and coordinate transformations, and on a reliable SLAM stack for low-level execution. Although the controlled depth-noise study shows only a 2.8-point SR decrease under the strongest tested perturbation, severe depth errors, accumulated odometry or SLAM drift, localization failure, and highly dynamic environments can cause larger degradation.

GRPO training relies on privileged simulator signals for progress, success, SPL, and collision, and the current system does not support online reinforcement learning on a physical robot. Real-world navigation rollouts are substantially slower than simulation, while computing the complete reward online would require endpoint annotations, human feedback, or a reliable learned verifier. Physical deployment is therefore limited to zero-shot transfer of the frozen policy learned in simulation.

The key-node mining heuristic selects CoT supervision using semantic similarity and visual change. It may under-select critical states when decision points are visually non-salient or when instruction styles differ substantially from those in R2R-CE and RxR-CE. Such settings may require additional training examples or a learned triggering criterion.

The current STI token encodes planar position (x, y) and orientation but does not include height and therefore cannot explicitly distinguish floors. Task-progress descriptions retained in memory anchors provide partial compensation but do not replace explicit 3D localization. Extending STI to a full 3D pose representation may improve robustness in more complex multi-floor buildings and uneven terrain. In addition, the reasoning-density reward uses fixed CoT-ratio thresholds of 0.4, where the density bonus begins to decay more steeply, and 0.6, where it reaches zero. These thresholds may require retuning when the useful reasoning density differs substantially from that of R2R-CE and RxR-CE.

6 Discussion and Conclusion

In this paper, we propose **TAMP-Nav**, a unified framework that achieves spatial, cognitive, and optimization alignment for embodied navigation. By introducing the **Pixel-to-3D Action Formulation**, an integrated **Selective Reasoning and Anchor-Trajectory Memory** mechanism, and a **Two-Level GRPO** paradigm, we

seamlessly bridge high-level visual-language reasoning with low-level physical execution. This approach advances VLM-based navigators while maintaining high sample and inference efficiency. Although currently trained exclusively on VLN-CE, future work will extend this framework to broader embodied tasks and dynamic real-world environments.

References

- Dong An, Hanqing Wang, Wenguan Wang, Zun Wang, Yan Huang, Keji He, and Liang Wang. 2024. Etpnav: Evolving topological planning for vision-language navigation in continuous environments. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Peter Anderson, Angel Chang, Devendra Singh Chaplot, Alexey Dosovitskiy, Saurabh Gupta, Vladlen Koltun, Jana Kosecka, Jitendra Malik, Roozbeh Mottaghi, Manolis Savva, and 1 others. 2018. On evaluation of embodied navigation agents. *arXiv preprint arXiv:1807.06757*.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, and 8 others. 2025. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*.
- Jiaqi Chen, Bingqian Lin, Ran Xu, Zhenhua Chai, Xiaodan Liang, and Kwan-Yee Wong. 2024. Mapgpt: Map-guided prompting with adaptive path planning for vision-and-language navigation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9796–9810.
- Kevin Chen, Junshen K Chen, Jo Chuang, Marynel Vázquez, and Silvio Savarese. 2021. Topological planning with transformers for vision-and-language navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11276–11286.
- Peihao Chen, Dongyu Ji, Kunyang Lin, Runhao Zeng, Thomas Li, Mingkui Tan, and Chuang Gan. 2022. Weakly-supervised multi-granularity map learning for vision-and-language navigation. *Advances in Neural Information Processing Systems*, 35:38149–38161.
- An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Zaitian Gongye, Xueyan Zou, Jan Kautz, Erdem Bıyık, Hongxu Yin, Sifei Liu, and Xiaolong Wang. 2024. Navila: Legged robot vision-language-action model for navigation. *arXiv preprint arXiv:2412.04453*.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, and 1 others. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, and 1 others. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
- Mengfei Du, Binhao Wu, Zejun Li, Xuan-Jing Huang, and Zhongyu Wei. 2024. Embspatial-bench: Benchmarking spatial understanding for embodied tasks with large vision-language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 346–355.
- Georgios Georgakis, Karl Schmeckpeper, Karan Wanchoo, Soham Dan, Eleni Miltsakaki, Dan Roth, and Kostas Daniilidis. 2022. Cross-modal map learning for vision and language navigation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15460–15470.
- Mobin Habibpour and Fatemeh Afghah. 2025. Think, remember, navigate: Zero-shot object-goal navigation with vlm-powered reasoning. *arXiv preprint arXiv:2511.08942*.
- Yicong Hong, Zun Wang, Qi Wu, and Stephen Gould. 2022. Bridging the gap between learning in discrete and continuous environments for vision-and-language navigation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15439–15449.
- Drew A Hudson and Christopher D Manning. 2019. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709.
- Jacob Krantz, Aaron Gokaslan, Dhruv Batra, Stefan Lee, and Oleksandr Maksymets. 2021. Waypoint models for instruction-guided navigation in continuous environments. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15162–15171.

- Jacob Krantz, Erik Wijmans, Arjun Majumdar, Dhruv Batra, and Stefan Lee. 2020. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In *European Conference on Computer Vision*, pages 104–120. Springer.
- Alexander Ku, Peter Anderson, Roma Patel, Eugene Ie, and Jason Baldridge. 2020. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4392–4412.
- Jiangyang Li, Cong Wan, SongLin Dong, Chenhao Ding, Qiang Wang, Zhiheng Ma, and Yihong Gong. 2026a. Trajectory-diversity-driven robust vision-and-language navigation. *arXiv preprint arXiv:2603.15370*.
- Zerui Li, Hongpei Zheng, Fangguo Zhao, Aidan Chan, Jian Zhou, Sihao Lin, Shijie Li, and Qi Wu. 2026b. One agent to guide them all: Empowering mllms for vision-and-language navigation via explicit world representation. *arXiv preprint arXiv:2602.15400*.
- Bingqian Lin, Yunshuang Nie, Ziming Wei, Jiaqi Chen, Shikui Ma, Jianhua Han, Hang Xu, Xiaojun Chang, and Xiaodan Liang. 2025. Navcot: Boosting llm-based vision-and-language navigation via learning disentangled reasoning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Jiahang Liu, Tianyu Xu, Jiawei Chen, Lu Yue, Jiazhao Zhang, Zhiyong Wang, Minghan Li, Qisheng Zhao, Anqi Li, Qi Su, and 1 others. 2026. Span-nav: Generalized spatial awareness for versatile vision-language navigation. *arXiv preprint arXiv:2603.09163*.
- Yuxing Long, Wenzhe Cai, Hongcheng Wang, Guanqi Zhan, and Hao Dong. 2024. Instructnav: Zero-shot system for generic instruction navigation in unexplored environment. *arXiv preprint arXiv:2406.04882*.
- Leland McInnes, John Healy, and James Melville. 2018. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.
- Zhangyang Qi, Zhixiong Zhang, Yizhou Yu, Jiaqi Wang, and Hengshuang Zhao. 2025. Vln-r1: Vision-language navigation via reinforcement fine-tuning. *arXiv preprint arXiv:2506.17221*.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, and 1 others. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmlR.
- Sonia Raychaudhuri, Saim Wani, Shivansh Patel, Unnat Jain, and Angel Chang. 2021. Language-aligned waypoint (law) supervision for vision-and-language navigation in continuous environments. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 4018–4028.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. 2023. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2998–3009.
- Ilias Stogiannidis, Steven McDonagh, and Sotirios A Tsaftaris. 2025. Mind the gap: Benchmarking spatial reasoning in vision-language models. *arXiv preprint arXiv:2503.19707*.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.
- Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. 2024. Eyes wide shut? exploring the visual shortcomings of multimodal llms. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9568–9578.
- Shaoan Wang, Yuanfei Luo, Xingyu Chen, Aocheng Luo, Dongyue Li, Chang Liu, Sheng Chen, Yangang Zhang, and Junzhi Yu. 2026a. Vlingnav: Embodied navigation with adaptive reasoning and visual-assisted linguistic memory. *arXiv preprint arXiv:2601.08665*.
- Teng Wang, Xinxin Zhao, Wenzhe Cai, and Changyin Sun. 2026b. Imaginenav++: Prompting vision-language models as embodied navigator through scene imagination. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

- Yunheng Wang, Yuetong Fang, Taowen Wang, Yixiao Feng, Yawen Tan, Shuning Zhang, Peiran Liu, Yiding Ji, and Renjing Xu. 2025. Dreamnav: A trajectory-based imaginative framework for zero-shot vision-and-language navigation. *arXiv preprint arXiv:2509.11197*.
- Zihan Wang, Xiangyang Li, Jiahao Yang, Yeqi Liu, and Shuqiang Jiang. 2023a. Gridmm: Grid memory map for vision-and-language navigation. In *Proceedings of the IEEE/CVF International conference on computer vision*, pages 15625–15636.
- Zihan Wang, Xiangyang Li, Jiahao Yang, Yeqi Liu, and Shuqiang Jiang. 2024. Sim-to-real transfer via 3d feature fields for vision-and-language navigation. *arXiv preprint arXiv:2406.09798*.
- Zun Wang, Jialu Li, Yicong Hong, Yi Wang, Qi Wu, Mohit Bansal, Stephen Gould, Hao Tan, and Yu Qiao. 2023b. Scaling data generation in vision-and-language navigation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12009–12020.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Meng Wei, Chenyang Wan, Jiaqi Peng, Xiqian Yu, Yuqiang Yang, Delin Feng, Wenzhe Cai, Chenming Zhu, Tai Wang, Jiangmiao Pang, and 1 others. 2025a. Ground slow, move fast: A dual-system foundation model for generalizable vision-and-language navigation. *arXiv preprint arXiv:2512.08186*.
- Meng Wei, Chenyang Wan, Xiqian Yu, Tai Wang, Yuqiang Yang, Xiaohan Mao, Chenming Zhu, Wenzhe Cai, Hanqing Wang, Yilun Chen, and 1 others. 2025b. Streamvln: Streaming vision-and-language navigation via slowfast context modeling. *arXiv preprint arXiv:2507.05240*.
- Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. 2022. Fast-lid2: Fast direct lidar-inertial odometry. *IEEE Transactions on Robotics*, 38(4):2053–2073.
- Fan Yang, Chao Cao, Hongbiao Zhu, Jean Oh, and Ji Zhang. 2022. Far planner: Fast, attemptable route planner using dynamic visibility update. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9–16. IEEE.
- Jiahui Zhang, Yurui Chen, Yanpeng Zhou, Yueming Xu, Ze Huang, Jilin Mei, Junhui Chen, Yu-Jie Yuan, Xinyue Cai, Guowei Huang, and 1 others. 2025a. From flatland to space: Teaching vision-language models to perceive and reason in 3d. *arXiv preprint arXiv:2503.22976*.
- Jiazhao Zhang, Anqi Li, Yunpeng Qi, Minghan Li, Jiahang Liu, Shaoan Wang, Haoran Liu, Gengze Zhou, Yuze Wu, Xingxing Li, and 1 others. 2025b. Embodied navigation foundation model. *arXiv preprint arXiv:2509.12129*.
- Jiazhao Zhang, Kunyu Wang, Shaoan Wang, Minghan Li, Haoran Liu, Songlin Wei, Zhongyuan Wang, Zhizheng Zhang, and He Wang. 2024a. Uni-navid: A video-based vision-language-action model for unifying embodied navigation tasks. *arXiv preprint arXiv:2412.06224*.
- Jiazhao Zhang, Kunyu Wang, Rongtao Xu, Gengze Zhou, Yicong Hong, Xiaomeng Fang, Qi Wu, Zhizheng Zhang, and He Wang. 2024b. Navid: Video-based vlm plans the next step for vision-and-language navigation. *arXiv preprint arXiv:2402.15852*.
- Lingfeng Zhang, Xiaoshuai Hao, Qinwen Xu, Qiang Zhang, Xinyao Zhang, Pengwei Wang, Jing Zhang, Zhongyuan Wang, Shanghang Zhang, and Renjing Xu. 2025c. Mapnav: A novel memory representation via annotated semantic maps for vlm-based vision-and-language navigation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13032–13056.
- Yue Zhang, Ziqiao Ma, Jialu Li, Yanyuan Qiao, Zun Wang, Joyce Chai, Qi Wu, Mohit Bansal, and Parisa Kordjamshidi. 2024c. Vision-and-language navigation today and tomorrow: A survey in the era of foundation models. *arXiv preprint arXiv:2407.07035*.
- Xinxin Zhao, Wenzhe Cai, Likun Tang, and Teng Wang. 2024. Imaginenav: Prompting vision-language models as embodied navigator through scene imagination. *arXiv preprint arXiv:2410.09874*.

Appendix

A Detailed Reward Formulations

A.1 Local Step Rewards

Based on our algorithm implementation, we define five unscaled reward components $r_i^{(t)}$ for a given state s_t and action a_t :

1. **Target Approach Reward ($r_{app}^{(t)}$)**: Let d_t be the expert geodesic distance. We calculate the distance reduction $\Delta d = d_t - d_{t+1}$ and the reward is then mapped via a signed sigmoid:

$$r_{app}^{(t)} = \frac{2}{1 + \exp(-\Delta d)} - 1 \quad (11)$$

2. **Collision Avoidance Reward ($r_{coll}^{(t)}$)**: Based on the distance to the nearest obstacle c_t , which can be directly queried from the Habitat simulator, we define a clamped reward:

$$r_{coll}^{(t)} = \max(0, \min(c_t, 1.0)) \quad (12)$$

3. **Stop Action Reward ($r_{stop}^{(t)}$)**: It encourages stopping near the goal while penalizing premature stops or moving away from the goal in the vicinity of the target:

$$r_{stop}^{(t)} = \begin{cases} 1.0, & \text{if } a_t = \text{STOP and } d_t < 1.5 \\ -1.0, & \text{if } a_t = \text{STOP and } d_t > 3.0 \\ 0.5, & \text{if } 1.5 \leq d_t < 2.5 \text{ and } (a_t = \text{STOP or } \Delta d \geq 0) \\ -0.5, & \text{if } 1.5 \leq d_t < 2.5 \text{ and } \Delta d < 0 \\ 0.0, & \text{otherwise} \end{cases} \quad (13)$$

4. **Reasoning Value Reward ($r_{rea}^{(t)}$)**: This component evaluates the specific contribution of Reasoning (CoT) to the current navigation progress. We define the reward based on the **performance margin** of the selected action's approach reward $r_{app}^{(t)}$ over the local baseline. Let $\bar{r}_{app}^{(t)} = \frac{1}{M} \sum_{j=1}^M r_{app}^{(t,j)}$ be the mean approach reward across all M sampled candidates at step t . The reward is formulated as:

$$r_{rea}^{(t)} = h_t \cdot (r_{app}^{(t)} - \bar{r}_{app}^{(t)}) \quad (14)$$

where $h_t \in \{0, 1\}$ is the CoT indicator at step t . This design ensures that the model is rewarded only when its reasoning leads to a decision that outperforms the average local policy, effectively penalizing redundant or misleading CoT.

5. **Format Adherence Reward ($r_{fmt}^{(t)}$)**: Ensures the model's output strictly follows the structured JSON format and valid spatial constraints:

$$r_{fmt}^{(t)} = \mathbb{I}_{valid} \cdot \rho_{bounds} \quad (15)$$

where $\mathbb{I}_{valid} \in \{0, 1\}$ indicates whether the parsed JSON matches the required schema, and $\rho_{bounds} = 0.0$ if the predicted pixel (u, v) falls outside the valid sensor range $[0, 279]^2$, otherwise 1.0.

A.2 Global Trajectory Rewards

The global reward evaluates the holistic quality of the completed trajectory, consisting of three components:

1. **Task Success Reward** (r_{suc}): The primary driver for navigation completion, relaxing strict success to oracle boundaries:

$$r_{suc} = \begin{cases} 1.0, & \text{if strictly successful} \\ 0.5, & \text{if oracle successful} \\ 0.0, & \text{otherwise} \end{cases} \quad (16)$$

2. **Trajectory Efficiency Reward** (r_{spl}): Optimizes navigation efficiency using Success weighted by Path Length (SPL), rewarding optimal trajectory execution:

$$r_{spl} = \text{SPL} \quad (17)$$

3. **Reasoning Density Reward** (r_{den}): Enforces a reasoning density constraint to prevent over-thinking. Let r be the ratio of reasoning steps in the sequence.

$$r_{den} = \begin{cases} 1.0 - r, & \text{if } 0 \leq r \leq 0.4 \\ 0.6 - 3.0(r - 0.4), & \text{if } 0.4 < r \leq 0.6 \\ 0.0, & \text{if } r > 0.6 \end{cases} \quad (18)$$

A.3 Loss Formulas

In our framework, the output at each decision step t (comprising Chain-of-Thought reasoning and action prediction) is a sequence of tokens. Let the generated sequence for the g -th trajectory at step t be $a_{g,t} = \{y_1, y_2, \dots, y_K\}$, where K is the number of tokens. our policy update is computed strictly at the **token level**.

Let $\rho_{g,t,k}$ denote the policy ratio for the k -th token at time step t :

$$\rho_{g,t,k} = \frac{\pi_{\theta}(y_k | C_{g,t}, y_{<k})}{\pi_{\theta_{old}}(y_k | C_{g,t}, y_{<k})} \quad (19)$$

where $C_{g,t}$ represents the historical context (including the initial instruction and previous observations), and $y_{<k}$ denotes the tokens generated before y_k within the current step.

The policy is optimized via the token-level GRPO objective. The step-level advantage $A_S^{(g,t)}$ is broadcast to all constituent tokens within that step:

$$L_{GRPO}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{g=1}^G \sum_{t=1}^{T_g} \frac{1}{K} \sum_{k=1}^K \left(\min \left(\rho_{g,t,k} A_S^{(g,t)}, \text{clip}(\rho_{g,t,k}, 1 - \epsilon, 1 + \epsilon) A_S^{(g,t)} \right) - \beta D_{KL}(\pi_{\theta}(\cdot | C_{g,t}, y_{<k}) || \pi_{ref}(\cdot | C_{g,t}, y_{<k})) \right) \right] \quad (20)$$

where T_g is the total number of decision steps in the g -th trajectory, and $A_S^{(g,t)}$ is the superimposed advantage (combining global and local rewards) corresponding to that specific step.

A.4 Hyperparameter Settings

In our implementation of the Two-Level GRPO framework, the hyperparameters for the reward formulations and the annealed guided sampling strategy are empirically set as follows:

Local Step Reward Weights: To balance fine-grained execution and reasoning, the weights are set to $\lambda_1 = 0.5$ (Target Approach), $\lambda_2 = 0.2$ (Collision Avoidance), $\lambda_3 = 0.8$ (Stop Action), $\lambda_4 = 0.5$ (Reasoning Value), and $\lambda_5 = 0.2$ (Format Adherence).

Global Trajectory Reward Weights: To strongly prioritize ultimate navigation task completion while maintaining trajectory efficiency and reasoning sparsity, the weights are set to $\omega_1 = 3.0$ (Task Success), $\omega_2 = 1.0$ (Trajectory Efficiency), and $\omega_3 = 1.0$ (Reasoning Density).

Annealed Guided Sampling: For the guidance coefficient $\beta_k = \beta_0 \cdot \alpha^k$, we set the initial value $\beta_0 = 2.0$ to ensure high-quality early rollouts and an exponential decay rate $\alpha = 0.99$ to gradually transition towards uniform exploration.

B Related Work

B.1 Action Formulation in VLM-based Navigation

Recent VLM-based navigators typically force the model to output predefined textual atomic actions (Zhang et al. (2024b), Cheng et al. (2024), Wei et al. (2025b)) or regress continuous 3D spatial coordinates (Zhang et al. (2025b)). However, recent analytical studies and benchmarks in multimodal perception expose a systematic representation gap: because foundational VLMs rely heavily on 2D contrastive vision encoders (e.g., CLIP) and image-text pairs, they suffer from profound shortcomings in explicit 3D spatial and geometric reasoning (Stogiannidis et al. (2025); Tong et al. (2024); Du et al. (2024)). Forcing these 2D-native models to predict unnatural 3D coordinates or rigid atomic steps directly contradicts their pre-training priors, often resulting in severe geometric hallucinations and poor sample efficiency. To mitigate this, recent literature argues that tightly coupled designs severely limit system performance, advocating for the decoupling of high-level semantic planning from low-level spatial execution (Li et al. (2026b)). However, this method requires the continuous, asynchronous construction of 3D metric and topological maps, incurring massive additional overhead. Other decoupling approaches attempt to simplify the VLM’s role into a multiple-choice selection by pre-computing navigable trajectories or employing diffusion models to generate future observations (Zhao et al. (2024); Wang et al. (2025, 2026b)). However, this not only introduces substantial computational overhead but also restricts the agent’s exploration to predefined heuristic proposals.

Sharing this decoupled philosophy, **TAMP-Nav** introduces a **Pixel-to-3D** action space. By prompting the VLM to act merely as an intuitive 2D “pointer” in the image plane, projecting the selected pixel into a 3D spatial coordinate, and offloading low-level execution to a deterministic SLAM controller, our paradigm seamlessly aligns embodied execution with the VLM’s inherent 2D visual capabilities, largely bypassing the 2D-to-3D representation gap.

B.2 Reasoning and Memory in VLM-based Navigation

Existing navigation methods often struggle to balance reasoning depth with memory efficiency over long horizons. On one hand, methods incorporating Chain-of-Thought (CoT) (Song et al. (2023); Lin et al. (2025); Liu et al. (2026); Habibpour and Afghah (2025)) enforce rigid, dense reasoning at every step, causing linear surges in inference latency. While a recent method, V LingNav (Wang et al. (2026a)), proposes adaptive reasoning to address this issue, it relies on coarse data annotations and lacks an in-depth exploration of how to systematically inject this meta-cognitive capability into the model. Simultaneously, memory designs like compressing historical frames into sparse tokens (Zhang et al. (2024b)) or relying on text-based maps (Chen et al. (2024)) inevitably lead to topological discontinuity or modal information loss. Although recent state-of-the-art models such as StreamVLN (Wei et al. (2025b)) and NavFoM (Zhang et al. (2025b)) have introduced 3D spatial voxelization or implicit forgetting curves to handle memory budgets, these compression and forgetting mechanisms may inadvertently discard critical historical details. Furthermore, they fundamentally lack explicit logical reasoning capabilities, resulting in significant performance degradation on complex, long-horizon tasks.

TAMP-Nav resolves these dilemmas via **Selective Reasoning and Anchor-Trajectory Memory**. By triggering deep CoT reasoning only at critical decision nodes and compressing routine traversed paths into lightweight spatiotemporal tokens, our framework strictly bounds the computational budget while maintaining global topological awareness and logical coherence across long horizons.

B.3 Reinforcement Learning for VLM-based Navigation

To alleviate the covariate shift in Imitation Learning, recent works apply GRPO to embodied navigation. However, they face two extremes: either relying exclusively on extreme sparse trajectory-level rewards (Li et al. (2026a)), which exacerbates credit assignment difficulties; or over-relying on strict expert action matching (Qi et al. (2025)), which restricts open-ended exploration. **TAMP-Nav** breaks this bottleneck with a **Two-Level GRPO** paradigm, superimposing global terminal advantages with local step advantages to provide dense supervision without relying on step-by-step expert-forcing.

C Algorithm for Key Node Selection

To ensure the reasoning process is only triggered at critical waypoints rather than at every step, we utilize a Spatiotemporal Key Node Mining strategy during the construction of the MultiNav-CoT dataset. The detailed algorithmic process is summarized in Algorithm 1. It consists of visual-semantic importance scoring, distance-based greedy filtering to maintain reasoning sparsity, and temporal in-filling to preserve global topological connectivity. In our implementation, we empirically set the spatial thresholds $D_{min} = 2$ and $D_{max} = 6$ to balance node sparsity with trajectory coverage.

To ensure a balanced contribution from both modalities in the combined importance score $S(t) = S_{sem}(t) + S_{vis}(t)$, we explicitly map both the semantic relevance and visual transition metrics to the strict $[0, 1]$ interval via trajectory-level Min-Max normalization. Let $E_{text}(\cdot)$ and $E_{vis}(\cdot)$ denote the pre-trained CLIP text and image encoders, respectively.

Semantic Importance Score (S_{sem}): The raw semantic score measures the cosine similarity between the instruction Q and the current visual observation v_t . The normalized score $S_{sem}(t)$ is defined as:

$$s_{sem}^{raw}(t) = \frac{E_{text}(Q) \cdot E_{vis}(v_t)}{\|E_{text}(Q)\| \|E_{vis}(v_t)\|} \quad (21)$$

$$S_{sem}(t) = \frac{s_{sem}^{raw}(t) - \min_{j \in [1, T]} s_{sem}^{raw}(j)}{\max_{j \in [1, T]} s_{sem}^{raw}(j) - \min_{j \in [1, T]} s_{sem}^{raw}(j)} \quad (22)$$

Visual Transition Score (S_{vis}): To capture significant scene transitions (e.g., turning a corner or entering a new room), the raw visual score measures the feature difference (calculated as $1 - \text{cosine similarity}$) between the current frame v_t and the previous frame v_{t-1} . For the first frame ($t = 1$), $s_{vis}^{raw}(1)$ is defined as 0. The normalized score $S_{vis}(t)$ is formulated as:

$$s_{vis}^{raw}(t) = 1 - \frac{E_{vis}(v_t) \cdot E_{vis}(v_{t-1})}{\|E_{vis}(v_t)\| \|E_{vis}(v_{t-1})\|} \quad (23)$$

$$S_{vis}(t) = \frac{s_{vis}^{raw}(t) - \min_{j \in [1, T]} s_{vis}^{raw}(j)}{\max_{j \in [1, T]} s_{vis}^{raw}(j) - \min_{j \in [1, T]} s_{vis}^{raw}(j)} \quad (24)$$

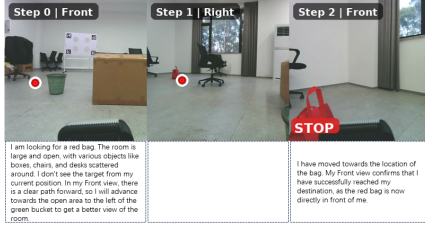
By independently standardizing $S_{sem}(t)$ and $S_{vis}(t)$ within each trajectory, they inherently share identical scales, ensuring a balanced contribution to the final selection objective $S(t)$.

D Real-World Experiment Details

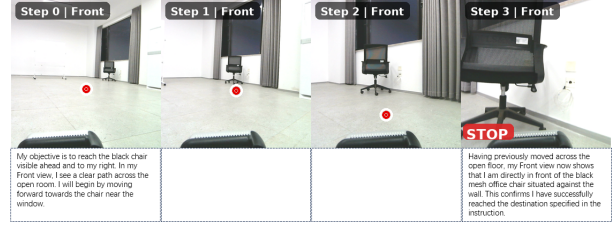
Hardware and System Setup. Our deployment platform is a Unitree Go2 quadruped robot, equipped with 4 annularly distributed RGB-D cameras (each providing an approximately 100° Field of View to collectively ensure 360° coverage) and a Hesai XT16 LiDAR mounted underneath. To bridge the sim-to-real gap, the system architecture adopts a decoupled design:

- (1) **High-Level Perception:** TAMP-Nav runs on a remote server, processing real-time multi-view images from the cameras and generating 2D pixel coordinates as navigation targets.
- (2) **Low-Level Execution:** The system utilizes the aligned depth measurements from the selected RGB-D camera to directly project the VLM-selected pixels into 3D world coordinates. It employs FastLIO (Xu et al. (2022)) for ego-motion estimation and utilizes FAR Planner (Yang et al. (2022)) for local path planning and dynamic obstacle avoidance.

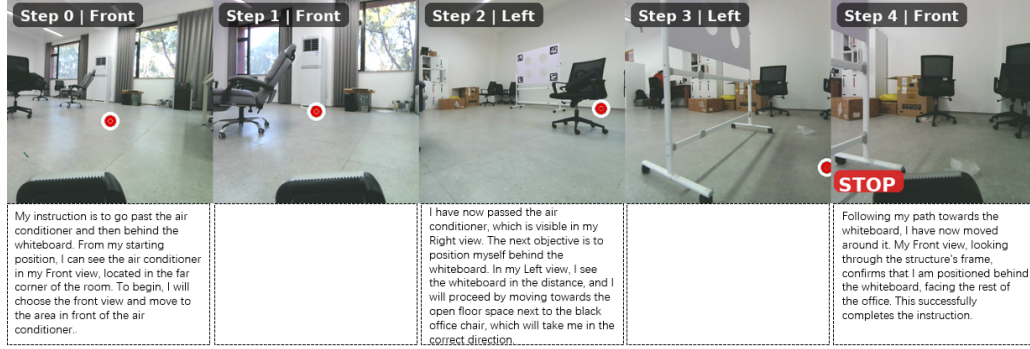
This decoupled Pixel-to-3D point selection design ensures that the VLM focuses on high-level semantic localization and reasoning, while intrinsic odometry, path execution, and motion safety are delegated to a mature classical SLAM stack.



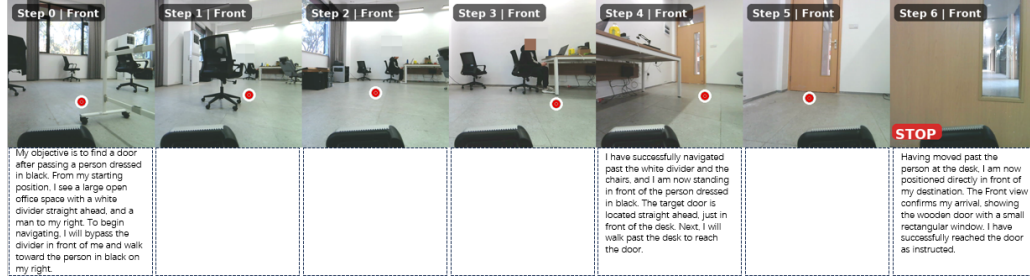
(a) Instruction: “Find the red bag”



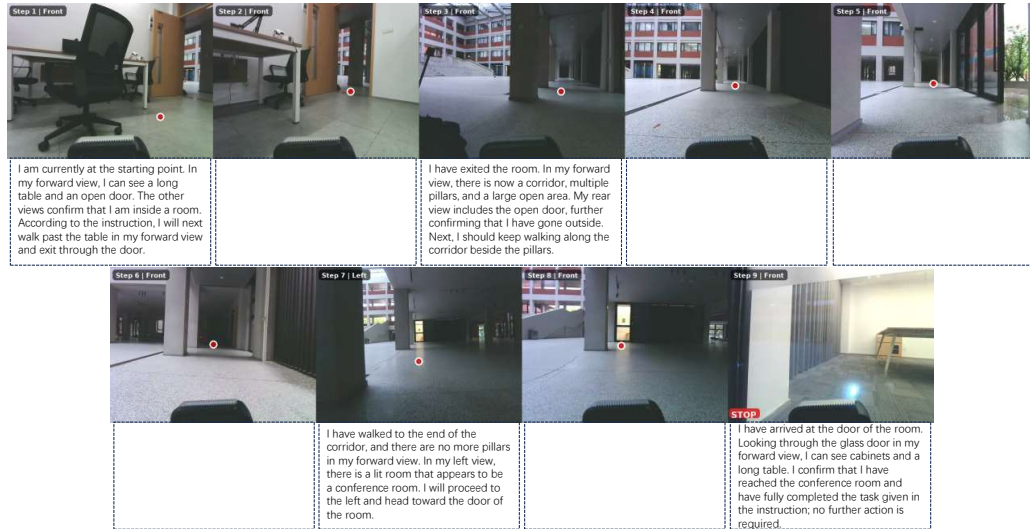
(b) Instruction: “Maps to the black chair on the right.”



(c) Instruction: “Proceed past the air conditioner and navigate behind the whiteboard.”



(d) Instruction: “Walk past the person in black and reach the door.”



(e) Instruction: “Go out the door, walk along the corridor next to the pillars to the end, then turn left and go to the entrance of the lit conference room.”

Figure 7 Examples of real-world experiments. The agent demonstrates zero-shot transferability across physical environments, executing multi-turn instructions by reasoning and outputting sequential 2D pixel coordinates.

Algorithm 1 Key Node Selection Strategy

Input: Trajectory $\mathcal{T}$ (with spatial poses p_t and visual frames v_t), Instruction Q , Spatial Thresholds D_{min}, D_{max}
Output: Key Node Set $\mathcal{K}$
// 1. Importance Scoring (Vectorized & Normalized)
Compute $S(t) \leftarrow S_{sem}(t) + S_{vis}(t)$ for all $t \in [1, T]$
// 2. Distance-based Greedy Filtering
 $\mathcal{I} \leftarrow \text{Argsort}(S)[0 : 0.3T]$ {Top-30% indices descending}
 $\mathcal{K} \leftarrow \emptyset$
for $t \in \mathcal{I}$ **do**
 if $\mathcal{K} = \emptyset \vee \min_{k \in \mathcal{K}} \text{dist}_{spatial}(p_t, p_k) \geq D_{min}$ **then**
 $\mathcal{K} \leftarrow \mathcal{K} \cup \{t\}$
 end if
end for
// 3. Temporal In-filling (Padding)
Sort $\mathcal{K}$ by time index
while $\exists$ adjacent pair (t_{curr}, t_{next}) in $\mathcal{K}$ s.t. $\text{dist}_{spatial}(p_{t_{curr}}, p_{t_{next}}) > D_{max}$ **do**
 $\mathcal{K} \leftarrow \mathcal{K} \cup \{\arg \max_{t \in (t_{curr}, t_{next})} S(t)\}$
 Sort $\mathcal{K}$ by time index
end while
return $\mathcal{K}$

Qualitative Analysis in Unseen Environments. Consistent with the evaluation protocol detailed in Section 4.6, we tested our framework in unmapped physical spaces featuring diverse objects, corridors, and obstacles. Figure 7 illustrates five representative execution trajectories. The agent demonstrates accurate object localization (Figure 7a and 7b) and effectively follows sequential instructions involving spatial relationships (Figure 7c and 7d). Furthermore, Figure 7e showcases an extended navigation task, confirming the model’s capability to process complex, multi-stage instructions over longer physical distances. As noted in the main text, TAMP-Nav is trained exclusively in simulation without real-world fine-tuning. The behavior in these unseen physical environments supports the zero-shot sim-to-real feasibility of our proposed paradigm.

E Analysis of STI token

To verify whether the trained positional encoder effectively retains pose information, we input the initial pose $p = (x, y, \text{yaw})$ into the positional encoder f_{pos} to obtain the corresponding STI token embedding:

$$e = f_{\text{pos}}(x, y, \text{yaw}) \in \mathbb{R}^d$$

where d denotes the output dimension of the encoder. In our implementation, this embedding serves as the unified pose representation processed by the subsequent VLM.

To visualize whether these high-dimensional embeddings preserve the underlying geometric structure of the original physical poses, we perform supervised dimensionality reduction. Specifically, we first randomly sample a large-scale set of poses and compute their corresponding embeddings. We then employ a clustering algorithm (McInnes et al. (2018)) to compress these embeddings into a 4-dimensional latent variable:

$$z = (z_x, z_y, z_s, z_c)$$

which is optimized to align with the normalized ground-truth components:

$$\left(\frac{x}{R}, \frac{y}{R}, \sin(\text{yaw}), \cos(\text{yaw}) \right)$$

where R is a constant representing the spatial range. Subsequently, the predicted pose $(\hat{x}, \hat{y}, \hat{\text{yaw}})$ is reconstructed from the latent variable (z_x, z_y, z_s, z_c) .

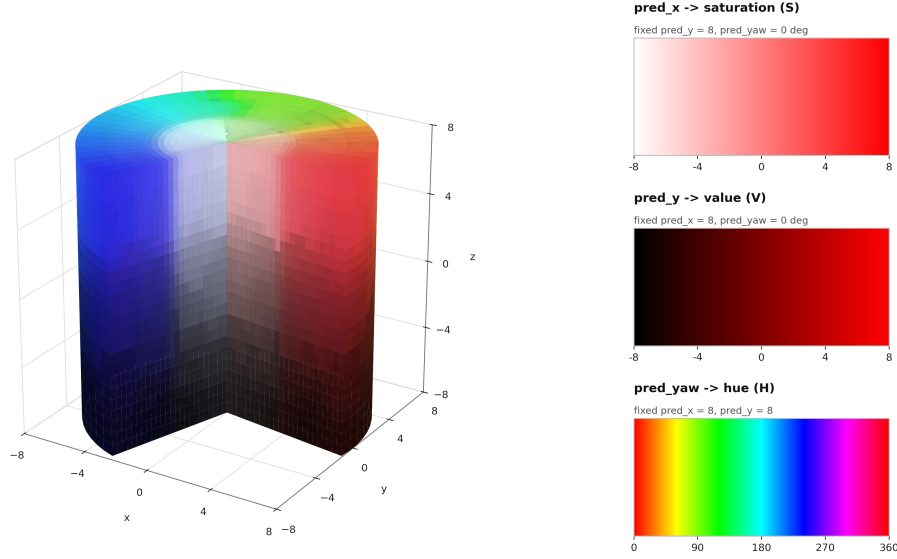


Figure 8 Visualization of STI token. We employ a clustering algorithm (McInnes et al. (2018)) to map high-dimensional embeddings into a 3D space.

As illustrated in Figure 8, we design a visualization scheme analogous to the HSV color space to simultaneously represent the three pose variables. In this geometric coordinate system, the circumferential direction corresponds to yaw, the radial distance from the center corresponds to x , and the vertical height corresponds to y . Consequently, the spatial position of each block in the figure is determined by its original ground-truth pose.

In parallel, the color of each block is mapped from its reconstructed pose: Hue (H) corresponds to $\hat{y}aw$, Saturation (S) to $\hat{x}$, and Value (V) to $\hat{y}$. If the positional encoder successfully preserves the pose information, the reconstructed color field should transition consistently with the spatial layout: hue should shift smoothly along the circumference, saturation along the radius, and value along the vertical axis. This visualization demonstrates that the "ground-truth poses define spatial positions, while poses recovered from embeddings define colors"; the high degree of alignment between the two confirms the fidelity of our positional encoder.

F Simulation Execution Example

To provide a more intuitive understanding of TAMP-Nav’s decision-making process, Figure 9 illustrates a representative navigation trajectory in the simulation environment.

Given the multi-step instruction: *“Go up the stairs and turn left. Wait at the doorway to the bedroom straight ahead.”*, the agent demonstrates robust visual-semantic grounding and logical planning. As shown in the figure, the model autonomously triggers its reasoning mechanism (CoT) at critical topological nodes—such as initially locating the stairs, determining the correct turning direction upon reaching the top floor, and confidently executing the stop action once the final destination is reached. This qualitative example highlights the effectiveness of our reasoning-on-demand and pixel-level point selection paradigm.

G Prompts for Structured CoT Generation

To enhance reasoning quality and mitigate hallucinations, TAMP-Nav employs a multi-stage prompt engineering pipeline. The tables below detail the core prompts utilized at each stage during the construction of the MultiNav-CoT dataset. **Please note that the content presented is a textually refined version. It is designed to clearly illustrate the core cognitive guidance and reasoning rules imposed on the model, and thus omits the complex JSON formatting validations and defensive directives found in the original engineering**



Figure 9 Visualization of a simulated navigation trajectory. The agent successfully executes the complex instruction: “Go up the stairs and turn left. Wait at the doorway to the bedroom straight ahead.” by triggering selective reasoning at key decision points and outputting point-and-click actions.

scripts.

Table 7 Pre-processing: Instruction Quality Filtering

System Prompt
You are a strict filter for navigation instructions.
- Pure action = a movement-only command without a target object/landmark/destination, e.g., "go straight", "turn left".
- Not pure action = includes a target or landmark, e.g., "go straight to the bed".
If there are 4 or more consecutive pure actions anywhere in the instruction, output DROP. Otherwise output KEEP. Consecutive means back-to-back pure actions with no non-pure-action instruction between them.
User Prompt Template
Instruction: {task}

Table 8 Stage 1: Task Phase Localization (History Summary)

System Prompt
You are an expert at summarizing navigation trajectories. Given a task and image sequences, your job is to summarize the trajectory from image $0 \rightarrow k$ in ONE continuous paragraph. Explicitly state which part of the task has been completed. Do NOT mention future parts. Return ONLY a JSON object.
User Prompt Template
Task: {task}
Summary ranges: {segment_list_text}
For EACH range $[0 \rightarrow k]$, write ONE paragraph in first person. Describe what happened and which part of the task is completed by that point.

Table 9 Stage 2: Current Observation Analysis

System Prompt
You are a controlled reasoning module. Describe the indoor scene layout (open paths, obstacles, landmarks) based on the attached front, left, back, and right views. Only describe visible objects. Do NOT mention targets, goals, history, or next steps.
Output Format
"Analyze the current observations": [your detailed description]

H Detailed Training Configurations

In this section, we provide the comprehensive training hyperparameters and experimental configurations for the Supervised Fine-Tuning (SFT) and the Two-Level GRPO stages of **TAMP-Nav**. All experiments were conducted on a computing cluster equipped with **8 NVIDIA A800 GPUs**, each with **80GB of VRAM**. Regarding computational cost, the Supervised Fine-Tuning (SFT) phase required approximately **160 GPU hours** (around 20 wall-clock hours) to complete, while the Two-Level GRPO phase took approximately **600 GPU hours** (around 75 wall-clock hours).

H.1 Supervised Fine-Tuning (SFT) Phase

During the SFT cold-start phase, we fine-tune the base Qwen2.5-VL-7B model to internalize the core navigation logic. To mitigate catastrophic forgetting of the model’s inherent general visual reasoning capabilities, we incorporate approximately 10% of `gqa_interleaved_cot` into our MultiNav-CoT training set. This dataset is a custom variant derived from the original GQA dataset ([Hudson and Manning \(2019\)](#)), augmented with interleaved Chain-of-Thought (CoT) annotations to align with our proposed paradigm.

The SFT is conducted for 1 epoch using the Bfloat16 (bf16) precision format. To optimize memory efficiency, we employ Gradient Checkpointing and Flash Attention 2. We utilize the AdamW optimizer with a cosine

Table 10 Stage 3: Future Action Reasoning (Infer Next Step)

System Prompt
IRON RULE: IF <code>gt_action == "stop"</code> , declare arrival. ELSE, reason step-by-step how the move advances the task. The final sentence MUST name the target view and an object-defined region. FORBIDDEN: coordinates, pixels, red circles, proportions.
User Prompt Template
Task: {task} History: {summary} Analysis: {analysis}
Target Guidance: {target_guidance} (Use this to locate the region, but do NOT mention the guidance itself).

Table 11 Stage 4: CoT Fusion and Polish

System Prompt
Rewrite the three-stage reasoning into a single, coherent English paragraph. Preserve spatial logic and the target view. Remove redundancies. Do NOT include stage titles. Output ONLY the polished paragraph.

learning rate scheduler and a warmup ratio of 0.03. The peak learning rate is set to 5×10^{-6} , with a weight decay of 0.01 and a maximum gradient norm of 5.0. The maximum sequence length is configured to 4096 tokens. The per-device training batch size is 1, with gradients accumulated over 32 steps to achieve a larger effective batch size.

H.2 Two-Level GRPO Phase

Following SFT, we apply the Two-Level GRPO framework to align the agent’s reasoning with physical execution. For the environmental setups and navigation instructions, we directly reuse the training dataset from the SFT phase. During this phase, the group size for trajectory-level rollouts is set to $G = 8$, and the candidate set size for step-level rollouts is $M = 4$. The maximum rollout trajectory length is capped at 24 steps.

For text generation during environmental exploration, we set the sampling temperature to 0.7 and top-p to 0.9 to encourage diverse trajectory sampling. The maximum prompt length is 4096 tokens, and the maximum completion length is 512 tokens. In the GRPO loss formulation, the KL divergence penalty coefficient (β) is set to 0.0.

The reinforcement learning (RL) phase is conducted for 800 training steps using bf16 precision. We continue to use the AdamW optimizer with a cosine scheduler, but with a reduced peak learning rate of 2×10^{-6} . The per-device training batch size is set to 2, with a gradient accumulation step of 32.